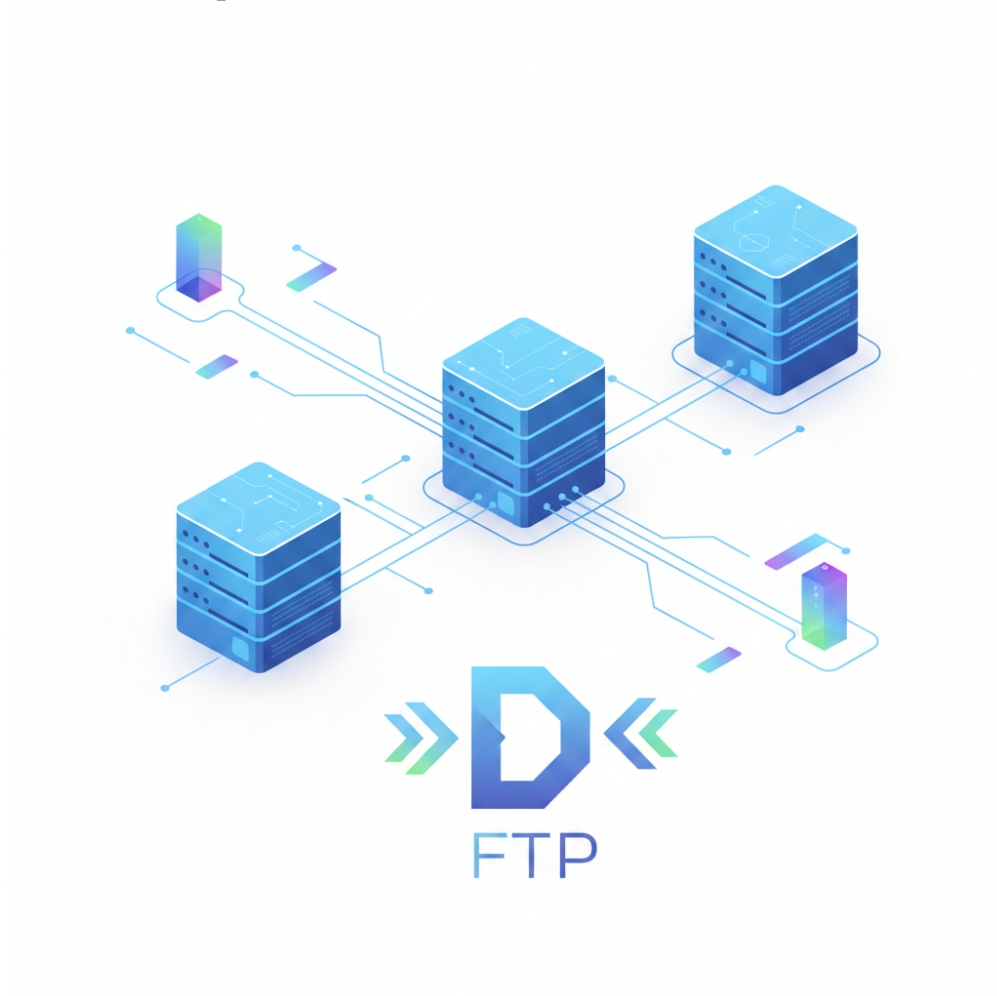


Informe Final del Proyecto

Diseño e Implementación de un Sistema Distribuido para Transferencia de Archivos FTP



Facultad de Matemática y Computación
Universidad de La Habana

29 de noviembre de 2025

Resumen

El presente informe describe el diseño e implementación de un sistema distribuido para la gestión de transferencia de archivos mediante el protocolo FTP, desarrollado como proyecto final de la asignatura Sistemas Distribuidos. El objetivo principal es transformar un servidor FTP centralizado, previamente funcional y compatible con la especificación RFC 959, en un sistema distribuido capaz de operar sobre una infraestructura basada en Docker Swarm y ejecutarse en dos máquinas físicas, garantizando tolerancia a fallos, disponibilidad y replicación de datos. El sistema propuesto adopta una arquitectura de clúster compuesta por tres tipos de nodos con roles claramente diferenciados: nodos *Router*, encargados de atender las sesiones FTP de los clientes; nodos de *Metadata*, responsables de la coordinación global, resolución de nombres, ubicación de datos y control de la replicación; y nodos de *Almacenamiento*, donde reside físicamente la información. Esta separación de responsabilidades permite escalar horizontalmente, mantener sesiones aisladas y delegar las operaciones críticas en un componente centralizado capaz de serializar accesos y decisiones globales. Para la tolerancia a fallos, el sistema adopta un modelo de replicación con un factor fijo de tres copias, distribuidas entre los nodos de almacenamiento. La coordinación entre componentes se realiza mediante RPC internos y mecanismos de bloqueo lógico gestionados por el servicio de Metadata, que además emplea un algoritmo de elección de líder sencillo para mantener una única autoridad activa incluso ante fallos. La comunicación con el cliente se mantiene íntegramente a través del protocolo FTP estándar, preservando compatibilidad sin modificar herramientas existentes. Este informe detalla la arquitectura del sistema, los procesos que lo componen, los mecanismos de comunicación, coordinación, nombrado, localización, consistencia, replicación, seguridad y tolerancia a fallos. Asimismo, se presentan las decisiones de diseño adoptadas, las limitaciones técnicas encontradas y las razones por las cuales se priorizó un enfoque simple y robusto sobre soluciones más complejas que exceden el alcance académico del proyecto.

Integrantes del Equipo

- | | |
|-----------------------------------|-------|
| 1. Javier Alejandro González Díaz | C-412 |
| 2. Kendry Javier del Pino Barbosa | C-412 |

Índice

1. Arquitectura y Diseño del Sistema	4
1.1. Organización general y Roles del Sistema Distribuido	4
1.2. Distribución de servicios en las redes de Docker	4
2. Procesos – ¿Cuántos programas/servicios hay?	5
2.1. Tipos de procesos	5
2.2. Agrupación de procesos en instancias	6
2.3. Patrón de diseño respecto al desempeño	6
3. Comunicación – ¿Cómo se envía información?	7
3.1. Tipo de comunicación	7
3.2. Comunicación cliente–servidor y servidor–servidor	8
3.3. Comunicación entre procesos	8
4. Coordinación – ¿Cómo se ponen de acuerdo los servicios?	9
4.1. Sincronización de acciones	9
4.2. Acceso exclusivo y condiciones de carrera	9
4.3. Toma de decisiones distribuidas	10
5. Nombrado y Localización – ¿Dónde está cada recurso?	12
5.1. Identificación de datos y servicios	12
5.2. Ubicación de datos y servicios	12
5.3. Localización (cómo llegar)	12
6. Consistencia y Replicación – ¿Qué pasa con varias copias?	13
6.1. Distribución de datos	13
6.2. Replicación y cantidad de réplicas	13
6.3. Confiabilidad tras una actualización	13
7. Tolerancia a fallas – ¿Qué pasa cuando algo se rompe?	14
7.1. Respuesta a errores	14
7.2. Nivel de tolerancia a fallos esperado	14
7.3. Fallos parciales, nodos caídos, nodos nuevos	15
8. Seguridad – ¿Qué tan vulnerable es el diseño?	15
8.1. Seguridad en la comunicación	15
8.2. Seguridad respecto al diseño	15
8.3. Autenticación y autorización	16

1. Arquitectura y Diseño del Sistema

1.1. Organización general y Roles del Sistema Distribuido

La idea central del sistema es construir un clúster lógico de servidores FTP formado por varios nodos con responsabilidades bien definidas, desplegados como contenedores en Docker Swarm y orquestados para trabajar de forma coordinada. Desde el punto de vista del usuario final, el sistema se presenta como un único servidor FTP estándar, pero internamente se comporta como un sistema distribuido tolerante a fallos.

Los nodos del sistema se pueden clasificar en las siguientes categorías principales:

- **Nodos Router/Front-end (R):** Son los puntos de entrada al sistema. Cada Router expone una interfaz FTP compatible con el estándar RFC 959 y atiende directamente a los clientes. Estos nodos no almacenan datos de forma persistente (a lo sumo mantienen caché temporal) y su responsabilidad principal es:
 - Autenticar a los usuarios.
 - Gestionar las sesiones FTP. Establecer y mantener las transferencias de datos hacia/desde los nodos de almacenamiento apropiados, actuando como proxy de datos.
 - Redirigir las solicitudes de transferencia de archivos a los nodos de almacenamiento adecuados.
- **Nodos de Metadata/ Coordinación (M):** Mantienen la vista global del sistema de archivos y del estado de las réplicas. Almacenan:
 - El espacio de nombres lógico (ejemplo: /user1/docs/file.txt).
 - El mapeo entre un identificador de fichero y el conjunto de nodos de almacenamiento que contienen sus réplicas.
 - Metadatos adicionales como tamaño, propietario, permisos y versión. Estos nodos forman un pequeño clúster replicado bajo un esquema de primario-copia (primary-backup): existe un nodo líder que procesa todas las operaciones de escritura y lectura de metadata, y uno o más nodos respaldo que mantienen copias actualizadas del estado. La lógica de consenso se mantiene deliberadamente simple para reducir la complejidad de implementación.
- **Nodos de Almacenamiento (S):** Son responsables del almacenamiento persistente de los archivos. Cada nodo S ejecuta un servidor FTP que gestiona el acceso a los datos en disco y atiende las operaciones de lectura y escritura que le son delegadas por los Routers. Los ficheros se almacenan de forma replicada: para cada archivo se mantienen varias copias distribuidas en diferentes nodos S, con el objetivo de garantizar la tolerancia a fallos y evitar pérdida de datos ante la caída de un nodo.

Nótese que a pesar de que en la implementación concreta ciertos roles podrían llegar a convivir en un mismo host físico, a nivel de diseño se tratan como componentes lógicos claramente separados.

1.2. Distribución de servicios en las redes de Docker

El despliegue del sistema se realiza sobre Docker Swarm utilizando redes overlay que separan el tráfico interno del sistema del tráfico con los clientes externos.

- **Red *frontend_net*:**
 - Conectada a los nodos Router y a la interfaz de red hacia el exterior.
 - Los clientes se conectan a un servicio Swarm (ftp.cluster) que balancea las conexiones FTP entre las distintas réplicas del Router.
 - Desde el punto de vista del cliente, existe un único servidor FTP.
- **Red *backend_net*:**
 - Conectada a los nodos Router, Metadata y Almacenamiento.

- Intercambio de mensajes de coordinación (heartbeats, notificaciones de fallo, etc.).
- Tráfico de datos FTP interno entre Routers y nodos de almacenamiento.

Nota: En el entorno físico disponible (dos máquinas), se plantea una distribución de servicios representativa:

■ **Máquina A:**

- 1 contenedor Router (R)
- 2 contenedores de almacenamiento (S)
- 1 contenedor de metadata/coordinación (M).

■ **Máquina B:**

- 1 contenedor Router (R)
- 2 contenedores de almacenamiento (S)
- 2 contenedores de metadata/coordinación (M).

Con esta configuración, todos los roles lógicos están presentes en ambos hosts, lo cual favorece la tolerancia a fallos, ya que la caída completa de una máquina no implica la desaparición total de un rol del sistema. A continuación un diagrama ilustrativo de la arquitectura del sistema distribuido abstraído de las limitaciones físicas:

2. Procesos – ¿Cuántos programas/servicios hay?

2.1. Tipos de procesos

El sistema se compone de tres tipos principales de procesos o servicios:

Proceso RouterFTP: Es la evolución del servidor FTP, adaptado al rol de Router.

- Acepta conexiones de control y datos FTP desde los clientes.
- Crea hilos o procesos hijos para atender múltiples clientes de forma simultánea.
- Implementa la lógica de traducción de comandos FTP a operaciones distribuidas.
- Incluye un cliente RPC/REST para comunicarse con los nodos de Metadata y resolver la ubicación de los archivos.

Proceso MetadataService: Es un servicio interno que ofrece una interfaz de consulta y actualización de metadata. Sus funciones incluyen:

- Resolver paths lógicos: `resolve(path) → [réplicas]`.
- Registrar la creación de nuevos ficheros: `createFile(path, size, user) → [réplicas]`.
- Actualizar el estado reportado por los nodos de almacenamiento: `reportReplica(nodeId, fileId, version)`.
- Mantener un log de operaciones que se replica hacia nodos de backup siguiendo un modelo primario-copia.

Este servicio no expone FTP, ya que su comunicación es exclusivamente interna.

Proceso StorageFTP: Ejecuta un servidor FTP convencional sobre el sistema de archivos local del contenedor de almacenamiento. Además:

- Incorpora un pequeño agente responsable de reportar periódicamente a Metadata qué ficheros y versiones están disponibles en ese nodo.
- Ejecuta las órdenes de replicación que Metadata le indique, copiando ficheros hacia o desde otros nodos S para mantener el factor de replicación deseado.

2.2. Agrupación de procesos en instancias

Desde el punto de vista de contenedores y servicios en Docker Swarm, se adopta la siguiente estrategia:

Separación por rol:

- **Servicio router:** instancias del proceso RouterFTP.
- **Servicio metadata:** instancias del proceso MetadataService.
- **Servicio storage:** instancias del proceso StorageFTP.

Esta separación permite escalar de forma independiente cada rol en función de la carga: por ejemplo, aumentar el número de Routers si crece el número de clientes concurrentes, o incrementar el número de nodos de almacenamiento si se requiere mayor capacidad.

En el contexto de la asignatura se evita fusionar roles en un mismo contenedor, priorizando una arquitectura clara y didáctica.

2.3. Patrón de diseño respecto al desempeño

A nivel de desempeño se aplican los siguientes patrones:

En los servidores FTP (RouterFTP y StorageFTP):

- Se utiliza un enfoque clásico de **multi-hilo o procesos concurrentes**: cada conexión de cliente puede ser atendida por un hilo dedicado. Esto permite atender múltiples clientes en paralelo y aprovechar mejor los recursos de CPU disponibles.

En el servicio de Metadata (MetadataService):

- Las operaciones críticas para la consistencia del sistema (creación de ficheros, cambios de permisos, modificaciones del espacio de nombres) se gestionan de forma **síncrona**, esperando confirmación de replicación hacia los nodos de backup antes de responder al Router.
- Las tareas de mantenimiento, como el re-balanceo de réplicas o la limpieza de entradas obsoletas, se ejecutan de manera **asíncrona** en segundo plano, evitando bloquear las operaciones principales.

Entre servicios (R ↔ M, M ↔ M, S ↔ S):

- La comunicación se realiza mediante **RPC sobre TCP**, típicamente con mensajes codificados en JSON sobre HTTP.
- Se definen **timeouts** y estrategias de **reintento** para manejar nodos temporalmente inaccesibles.
- Algunas operaciones, como la replicación masiva, se organizan en colas internas para ser procesadas de forma gradual, reduciendo picos de carga.

3. Comunicación – ¿Cómo se envía información?

3.1. Tipo de comunicación

En el sistema conviven cuatro tipos principales de comunicación:

Cliente ↔ Router / Storage:

- Se utiliza exclusivamente el protocolo **FTP clásico** (canal de control y canal de datos) conforme a la especificación RFC 959.
- Los clientes se conectan siempre a un Router; en el diseño adoptado, el Router actúa como **proxy de datos**, manteniendo el control completo del flujo entre cliente y almacenamiento.

Router ↔ Metadata:

- La comunicación se implementa mediante **RPC/REST sobre TCP** a través de la red **backend_net**.
- El Router envía peticiones al servicio de Metadata para:
 - Resolver rutas lógicas a conjuntos de réplicas.
 - Registrar nuevas operaciones sobre el espacio de nombres.
 - Consultar permisos y propietarios de los ficheros.

Metadata ↔ Metadata:

- Entre instancias de Metadata se intercambian **mensajes internos de coordinación y replicación de logs**.
- Estos mensajes incluyen, por ejemplo, actualizaciones de registro, *heartbeats* y notificaciones de cambio de estado.
- El esquema de **primario-copia** se apoya en este canal para mantener el estado replicado, evitando la complejidad de un algoritmo de consenso completo.

Metadata ↔ Storage:

- Entre los nodos de Metadata y los nodos de almacenamiento se establece un **canal de control** sobre TCP en la red **backend_net**, implementado mediante mensajes ligeros (RPC/REST).
- Desde Metadata hacia Storage se envían **órdenes de gestión de réplicas**, tales como:
 - Crear una nueva réplica de un fichero en un nodo S concreto.
 - Resincronizar una réplica desactualizada a partir de otra réplica más reciente.
 - Invalidar o marcar como obsoleta una réplica tras una reconfiguración.
- Desde Storage hacia Metadata se envían **mensajes de confirmación y reporte de estado**, que incluyen:
 - Confirmaciones de éxito o fallo de las órdenes anteriores (*ACK/NACK*) asociadas a un **FileID** y a una versión concreta.
 - Información periódica sobre el conjunto de ficheros almacenados localmente y sus versiones.
 - Indicadores de estado del nodo (espacio libre, disponibilidad, incidencias).
- Esta comunicación es asimétrica: Metadata toma las decisiones globales y emite órdenes, mientras que los nodos de almacenamiento ejecutan dichas órdenes y reportan su resultado, permitiendo a Metadata mantener una visión consistente del estado de las réplicas en el sistema.

Storage ↔ Storage:

- Para la replicación de datos entre nodos de almacenamiento se utiliza también **FTP** como protocolo de transferencia, de forma que un nodo S puede actuar como cliente FTP de otro nodo S en la red interna.
- Esta elección mantiene la **homogeneidad** del sistema en torno al protocolo FTP, simplifica la reutilización de código ya existente y respeta la idea de que todos los servidores relevantes hablan FTP, aunque esta comunicación sólo sea visible en la red de *backend*.

3.2. Comunicación cliente–servidor y servidor–servidor

Relación cliente–servidor:

- El cliente se conecta al nombre lógico del servicio, `ftp.cluster`, que está asociado en Docker Swarm a un conjunto de contenedores Router.
- El balanceador de Swarm distribuye las conexiones FTP entre las distintas instancias de Router.
- Para el usuario, la interacción es idéntica a la de un único servidor FTP tradicional.

Relación servidor–servidor:

- **R → M**: El Router envía peticiones RPC al servicio de Metadata para resolver nombres, consultar permisos y registrar nuevas operaciones del sistema de archivos.
- **R → S**: El Router establece conexiones FTP de datos hacia los nodos de almacenamiento para realizar las operaciones de lectura y escritura de ficheros, actuando como intermediario entre el cliente y el almacenamiento.
- **M → S**: Metadata envía órdenes de replicación, invalidación o sincronización a los nodos de almacenamiento, indicándoles qué ficheros deben copiar o actualizar.
- **M ↔ M**: Las instancias de Metadata intercambian mensajes para replicar el estado, detectar fallos entre sí y mantener la consistencia del clúster de control.

3.3. Comunicación entre procesos

- Dentro de cada contenedor, la comunicación entre hilos o procesos internos se realiza mediante los mecanismos clásicos del sistema operativo (memoria compartida, colas internas, estructuras de datos compartidas protegidas con primitivas de sincronización, etc.).
- Entre contenedores y nodos físicos, toda la comunicación se basa en **sockets TCP** sobre la red `backend_net`.
- Sobre estos sockets se definen mensajes de nivel superior, tales como:
 - PING / PONG para detección de *aliveness*.
 - REPLICATION_MISSING(fileId, version) para notificar inconsistencias en réplicas.
 - HEARTBEAT(nodeId, load, freeSpace) para informar del estado de un nodo.

Este diseño mantiene la comunicación simple, trazable y fácilmente depurable.

4. Coordinación – ¿Cómo se ponen de acuerdo los servicios?

4.1. Sincronización de acciones

Existen varias operaciones que requieren coordinación estricta para preservar la coherencia del sistema:

- Creación de nuevos ficheros.
- Borrado y renombrado de entradas en el espacio de nombres.
- Modificación de permisos y propietarios.

Para todas estas operaciones, el flujo es:

1. El Router envía la solicitud al **líder de Metadata**.
2. El líder registra la operación en su **log interno**.
3. El líder **replica** dicha entrada en los nodos de *backup*.
4. Una vez que la replicación se ha completado, el líder aplica la operación a su estado local y responde al Router.

De esta manera se garantiza que las modificaciones estructurales se procesen en un **orden global bien definido**, reduciendo las posibilidades de inconsistencias en el espacio de nombres.

4.2. Acceso exclusivo y condiciones de carrera

Para evitar **condiciones de carrera**, especialmente en operaciones de escritura concurrente sobre un mismo fichero o directorio, se adoptan dos niveles de control:

A nivel de Metadata:

- Se mantiene una **tabla de bloqueos lógicos**, por ejemplo `locks[path] = ownerRouterId`.
- Antes de permitir una escritura, el Router debe **adquirir un bloqueo** sobre el recurso en el nodo de Metadata.
- Mientras un **lock** esté activo, otras operaciones que compitan por el mismo recurso se bloquean o se rechazan hasta que el **lock** se libere.

A nivel de almacenamiento:

- Los servidores StorageFTP pueden apoyarse en mecanismos de *lock* a nivel de sistema de archivos o asumir que la exclusión ya fue garantizada por Metadata, simplificando su lógica.
- En cualquier caso, las escrituras concurrentes se gestionan de forma controlada para evitar corrupción de datos.

4.3. Toma de decisiones distribuidas

Diversas decisiones deben tomarse de manera distribuida:

Elección del nodo de Metadata líder:

- El subsistema de Metadata requiere una única autoridad que serialice los cambios del espacio de nombres y coordine la replicación de los ficheros. Para ello se utiliza un **algoritmo de elección de líder sencillo, basado en el algoritmo Bully**, que resulta adecuado para un entorno controlado y con pocos nodos.
- El funcionamiento general es el siguiente: ante la caída del líder o la ausencia de *heartbeats*, un nodo de Metadata inicia una elección enviando un mensaje a los nodos con identificadores mayores. Si recibe respuesta de alguno de ellos, cede la elección y espera a que un nodo de mayor prioridad complete el proceso. Si no recibe ninguna respuesta, se autoproclama **nuevo líder** y notifica su rol al resto de nodos.
- Este mecanismo proporciona una solución práctica para mantener la disponibilidad del servicio de Metadata sin introducir la complejidad de un protocolo de consenso completo como Raft, cuyo diseño y correcta implementación no son objetivos de este proyecto y exceden el alcance de la asignatura. La consistencia del estado de Metadata se garantiza mediante un esquema *primario-copia*, suficientemente robusto para las necesidades de esta arquitectura.

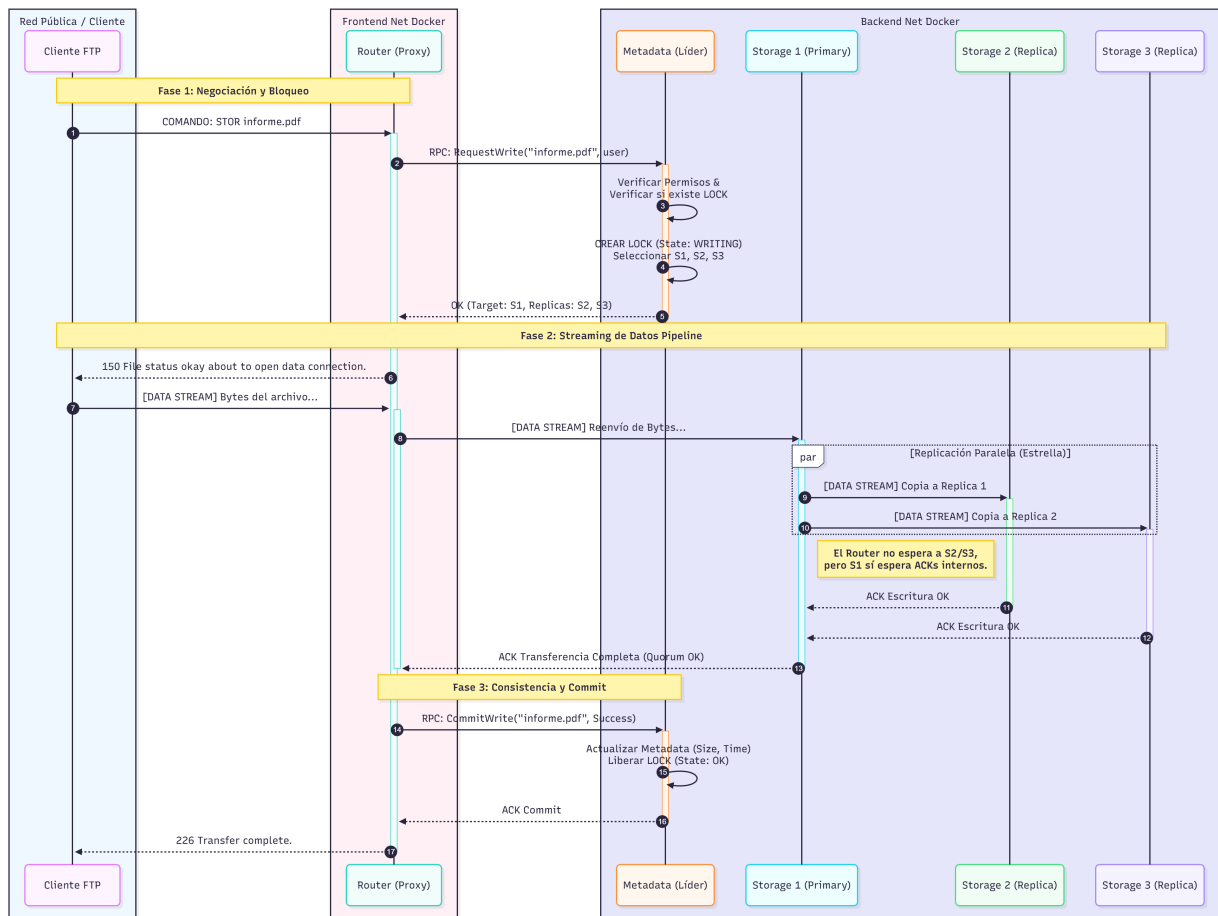
Selección de nodos de almacenamiento para nuevas réplicas:

- El líder de Metadata mantiene información sobre espacio disponible y carga de cada nodo S.
- Para cada nuevo fichero se eligen varias réplicas intentando:
 - Balancear la carga global.
 - Distribuir las réplicas entre ambos *hosts* físicos siempre que sea posible.

Reacción ante la caída de nodos:

- Si un nodo deja de enviar *heartbeats* durante un intervalo de tiempo determinado, se le marca como `\SUSPECT` y posteriormente como `\DOWN`.
- Metadata reconfigura internamente las listas de réplicas, evitando utilizar nodos marcados como inactivos.
- Cuando un nodo retorna, se sincroniza con el estado actual antes de volver a ser considerado completamente operativo.

A continuación un ejemplo ilustrativo del flujo de creación de un nuevo fichero, para ilustrar de forma simple la coordinación entre componentes:



5. Nombrado y Localización – ¿Dónde está cada recurso?

5.1. Identificación de datos y servicios

La identificación de los recursos sigue dos niveles:

Datos (ficheros): Cada fichero se identifica por:

- Un **path lógico** visible para el usuario, por ejemplo `/user1/docs/informe.pdf`.
- Un **identificador interno (FileID)**, que puede derivarse del *path* y un número de versión, y que se utiliza internamente para referenciar el archivo de forma única y estable.

Servicios (nodos del sistema):

- Cada nodo tiene un **nodeId lógico** y se asocia a un nombre de servicio en Docker Swarm (por ejemplo, `router1`, `storage2`).
- A partir del `nodeId` y el servicio de Swarm es posible obtener la **dirección de red y el puerto** correspondientes.

5.2. Ubicación de datos y servicios

Ubicación de datos:

- El servicio de Metadata mantiene una tabla principal donde se asocia cada `FileID` a la lista de réplicas: `FileID → (nodeId1, version), (nodeId2, version), ...`
- Esta información se actualiza cuando se crean nuevas réplicas, se eliminan nodos o se detectan inconsistencias.

Ubicación de servicios:

- Los servicios `router` y `storage` se exponen mediante el sistema de **resolución de nombres y balanceo de Docker Swarm**.
- Metadata puede descubrir y actualizar la lista de nodos activos mediante mecanismos de **registro estático y/o heartbeats**.

5.3. Localización (cómo llegar)

El siguiente flujo ilustra el mecanismo de localización durante la lectura de un fichero:

1. El cliente ejecuta el comando `RETR /user1/docs/informe.pdf` sobre el servidor FTP lógico.
2. El Router recibe el comando y traduce el *path* lógico a un `FileID`.
3. El Router consulta a Metadata mediante `resolve(FileID)`.
4. Metadata responde con una **lista de réplicas ordenada por prioridad**, por ejemplo `[S1, S2, S3]`.
5. El Router intenta establecer un canal de datos FTP con el nodo S de mayor prioridad (`S1`).
6. Si `S1` no está disponible, reintenta con `S2`, y así sucesivamente.

De este modo, el cliente no necesita conocer la existencia de múltiples nodos: la localización y selección de réplicas queda completamente **encapsulada** en la lógica del Router y del servicio de Metadata.

6. Consistencia y Replicación – ¿Qué pasa con varias copias?

6.1. Distribución de datos

El sistema no utiliza una DHT, sino una **distribución controlada centralmente** por el servicio de Metadata, lo que simplifica el razonamiento sobre el comportamiento ante fallos.

Cuando se crea un fichero, el líder de Metadata:

- Decide el conjunto de nodos de almacenamiento que lo almacenarán.
- Tiene en cuenta criterios como:
 - Espacio libre disponible.
 - Carga de cada nodo.
 - Distribución entre los dos *hosts* físicos para minimizar riesgo ante la caída completa de una máquina.

El objetivo es que cada fichero cuente con un número fijo de réplicas, distribuidas de forma equilibrada en el clúster.

6.2. Replicación y cantidad de réplicas

Para lograr un sistema **2-tolerante a fallos** a nivel lógico, se establece un factor de replicación **R = 3** para cada fichero. Esto significa que:

- Cada fichero se almacena en **tres nodos de almacenamiento distintos**.
- El sistema puede seguir ofreciendo acceso a los datos incluso si hasta dos réplicas de un fichero dejan de estar disponibles.

El protocolo de escritura adoptado es el siguiente:

1. El Router consulta a Metadata las réplicas asignadas para el fichero.
2. El Router envía la nueva versión del fichero a una réplica primaria (S_1).
3. S_1 almacena el fichero localmente e:
 - Inicia la replicación hacia S_2 y S_3 en *background*, confirmando al cliente tras un número mínimo de réplicas confirmadas (dos) para reducir la latencia.

En el contexto del proyecto se da prioridad a una implementación simple y comprensible del protocolo de replicación, aún cuando ello implique sacrificar ciertas garantías de consistencia fuerte exhaustiva.

6.3. Confiabilidad tras una actualización

Para garantizar que las réplicas convergen hacia un estado consistente:

- Cada fichero mantiene un **número de versión** que se incrementa con cada escritura.
- Cuando un nodo de almacenamiento se reincorpora tras una caída, se sincroniza con Metadata:
 - Envía su lista de (`FileID`, `localVersion`) para informar del estado local.
 - Metadata responde indicando qué ficheros están desactualizados o faltan.
 - El nodo descarga las versiones necesarias de otros nodos de almacenamiento actualizados.

En caso de partición de red, puede ocurrir que diferentes particiones acepten escrituras concurrentes sobre el mismo fichero. Para manejar estos escenarios se adopta una política simple:

- Se emplea un esquema de **last-write-wins** basado en números de versión. Requiere una implementación de relojes lógicos simples.
- Cuando Metadata detecta versiones incompatibles, conserva aquella considerada más reciente.

7. Tolerancia a fallas – ¿Qué pasa cuando algo se rompe?

7.1. Respuesta a errores

El diseño contempla distintos tipos de fallos:

- Caída de nodos de almacenamiento (S).
- Caída de nodos de metadata/coordinación (M).
- Caída de nodos Router (R).
- Particiones de red entre los dos *hosts* físicos.

Para responder a estos eventos, el sistema implementa los siguientes mecanismos:

Heartbeats periódicos:

- Cada nodo envía **señales de vida** al servicio de Metadata.
- La ausencia de *heartbeats* desencadena la transición a estados de **sospecha y caída**.

Reintentos y conmutación por error:

- Si un Router no puede contactar con una réplica primaria (S_1), intenta acceder a las **réplicas alternativas** (S_2, S_3).
- Esto mejora la disponibilidad percibida por el cliente.

Reconfiguración automática:

- Cuando un nodo de almacenamiento se considera definitivamente caído, Metadata planifica la creación de **nuevas réplicas** en otros nodos para restaurar el factor de replicación establecido.

7.2. Nivel de tolerancia a fallos esperado

El objetivo es alcanzar un nivel **2-tolerante a fallos por rol** en el modelo lógico:

Metadata:

- Se despliegan múltiples nodos M bajo un esquema **primario-copia**.
- La pérdida de uno o varios nodos de *backup* no impide que el líder siga funcionando, aunque su caída requiere una recuperación manual o semiautomática.

Storage:

- Con un factor de replicación de $R = 3$, un fichero puede seguir disponible mientras al menos **una réplica permanezca accesible**.
- Esto permite tolerar la caída simultánea de **hasta dos nodos** de almacenamiento que contengan dicho fichero.

Router:

- Al desplegar más de un Router, la caída de uno de ellos **no interrumpe el servicio**, ya que los clientes pueden seguir conectándose a las instancias restantes.

Limitación práctica: la caída completa de un *host* físico puede implicar la pérdida simultánea de varias réplicas. Por ello, se procura que las réplicas de un mismo fichero se **repartan entre los dos hosts**, reduciendo el riesgo.

7.3. Fallos parciales, nodos caídos, nodos nuevos

El sistema está preparado para gestionar situaciones dinámicas:

Nodos caídos temporalmente:

- Se marcan como **DOWN** en Metadata. Sus réplicas no se utilizan.
- Al regresar, el nodo entra en estado **RECOVERING**, **sincroniza su estado** con el clúster y, una vez actualizado, pasa a estar disponible (**UP**).

Nuevos nodos:

- Al incorporarse un nuevo nodo de almacenamiento, se registra en Metadata.
- Se le asignan **gradualmente nuevas réplicas** para contribuir al balance de carga y aumentar la capacidad.

Partición de red:

- Cada partición ve a los nodos del otro segmento como caídos.
- Se puede priorizar:
 - La **consistencia** (sólo la partición que mantiene el líder de Metadata aceptaría escrituras), o
 - La **disponibilidad** (permitiendo escrituras en ambas particiones y resolviendo conflictos al reconectar).
- Una vez restablecida la comunicación, los nodos de Metadata intercambian sus *logs* y estados, y los nodos de almacenamiento sincronizan las réplicas desactualizadas.

8. Seguridad – ¿Qué tan vulnerable es el diseño?

8.1. Seguridad en la comunicación

En el plano de la comunicación se consideran dos niveles:

Entre cliente y Routers (R):

- El canal candidato es **FTPS** (FTP sobre TLS) para proteger credenciales y datos en tránsito.
- En caso de no implementar FTPS, se justifica que el entorno de pruebas se opera en una red controlada y se señala explícitamente esta limitación como trabajo futuro.

Entre nodos internos (R, M, S) en backend.net:

- La red de *backend* se configura como una **red overlay privada**, no directamente accesible desde redes externas.
- Opcionalmente, las peticiones RPC pueden ir firmadas con un token compartido.

8.2. Seguridad respecto al diseño

El diseño del sistema incorpora varios principios básicos de seguridad:

Separación de roles y responsabilidades:

- La lógica de **autoridad** (decisiones sobre espacio de nombres, permisos y replicación) se concentra en el servicio de Metadata.
- Los nodos de almacenamiento se limitan a gestionar datos locales y a obedecer órdenes; **no deciden sobre la política global**.

Principio de mínimo privilegio:

- Cada nodo sólo tiene acceso al conjunto de recursos que necesita para desempeñar su función.
- Los procesos de almacenamiento sólo operan sobre su propio volumen de datos.
- Los Routers **no manipulan directamente el estado interno** de Metadata, sino que interactúan a través de interfaces bien definidas.

Persistencia confiable de Metadata:

- El estado del servicio de Metadata se **persiste en disco** en los nodos M, evitando que el reinicio de un proceso implique pérdida de información crítica.
- El uso de *logs* permite **reconstruir el estado tras fallos bruscos** (Backguard Recovery), posibilitando retroceder a un estado funcional, aumentando la robustez del sistema.

8.3. Autenticación y autorización

La autenticación y autorización se centralizan en el servicio de Metadata:

Autenticación:

- Los usuarios se identifican mediante credenciales (**usuario/contraseña**) transmitidas a través del protocolo FTP.
- El Router delega la **verificación de estas credenciales** al servicio de Metadata.

Autorización:

- Para cada fichero o directorio se almacenan **permisos asociados** (propietario, grupo, bits de acceso al estilo Unix).
- Ante operaciones sensibles (RETR, STOR, DELE, etc.), el Router consulta a Metadata si el usuario tiene los **permisos requeridos** antes de ejecutarlas.

Aislamiento entre usuarios:

- Cada usuario dispone de un **directorio “home” lógico**, y la vista del sistema de archivos se limita a aquellos recursos para los que posee permisos.
- Esto evita accesos indebidos a datos de otros usuarios o a zonas internas del sistema.

